

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100
#define MAX_TOKEN_LEN 256

char *expand_variable(const char *input)
{
    static char result[MAX_TOKEN_LEN];
    int i = 0, j = 0;

    while (input[i] != '\0' && j < MAX_TOKEN_LEN - 1)
    {
        if (input[i] == '$')
        {
            char variable[100];
            int k = 0;

            i++;

            while (isalnum(input[i]) || input[i] == '_' )
            {
                if (k < 99)
                    variable[k++] = input[i];

                i++;
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL)
            {
                for (int x = 0; value[x] != '\0' && j < MAX_TOKEN_LEN - 1; x++)
                {
                    result[j++] = value[x];
                }
            }
            else
            {
                result[j++] = input[i++];
            }
        }

        result[j] = '\0';

        return result;
    }
}
```

```

void parse_command(char *input)
{
    char tokens[MAX_TOKENS][MAX_TOKEN_LEN];
    int token_count = 0;

    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')
            break;

        int j = 0;

        /*
         * SINGLE QUOTES
         * Everything inside single quotes is treated literally.
         */
        if (input[i] == '"')
        {
            i++;

            while (input[i] != '\0' && input[i] != '"')
            {
                if (j < MAX_TOKEN_LEN - 1)
                    tokens[token_count][j++] = input[i];

                i++;
            }

            if (input[i] == '"')
                i++;

            tokens[token_count][j] = '\0';

            printf("Single quoted token: [%s]\n",
                tokens[token_count]);

            token_count++;
        }

        /*
         * DOUBLE QUOTES
         * Spaces are preserved, but variables are expanded.
         */
        else if (input[i] == '"')
        {
            char temp[MAX_TOKEN_LEN];
            int k = 0;

            i++;

            while (input[i] != '\0' && input[i] != '"')
            {
                if (k < MAX_TOKEN_LEN - 1)
                    temp[k++] = input[i];

                i++;
            }

```

```

    }

    if (input[i] == "")
        i++;

    temp[k] = '\0';

    char *expanded = expand_variable(temp);

    strcpy(tokens[token_count], expanded);

    printf("Double quoted token: [%s]\n",
        tokens[token_count]);

    token_count++;
}

/*
 * NORMAL TOKEN
 */
else
{
    while (input[i] != '\0' &&
        !isspace(input[i]) &&
        input[i] != '"' &&
        input[i] != '"')
    {
        if (j < MAX_TOKEN_LEN - 1)
            tokens[token_count][j++] = input[i];
        i++;
    }
    tokens[token_count][j] = '\0';

    printf("Normal token: [%s]\n",
        tokens[token_count]);

    token_count++;
}

if (token_count >= MAX_TOKENS)
    break;
}

printf("\nTotal tokens: %d\n", token_count);

printf("\nParsed tokens:\n");

for (int x = 0; x < token_count; x++)
{
    printf("Token %d = [%s]\n",
        x + 1,
        tokens[x]);
}
}

int main()
{
    char input[MAX_INPUT];

    /*
     * Set variables for testing.

```

```

*/
setenv("NAME", "Adithya", 1);
setenv("COURSE", "CSE", 1);
setenv("COLLEGE", "KL University", 1);

printf("=====\n");
printf("    QUOTED STRING PARSER\n");
printf("=====\n");

printf("\nEnvironment variables:\n");
printf("NAME = %s\n", getenv("NAME"));
printf("COURSE = %s\n", getenv("COURSE"));
printf("COLLEGE = %s\n", getenv("COLLEGE"));

printf("\nEnter a command:\n");

if (fgets(input, sizeof(input), stdin) == NULL)
{
    printf("Input error.\n");
    return 1;
}

input[strcspn(input, "\n")] = '\0';

printf("\n=====\n");
printf("Parsing Result\n");
printf("=====\n");

parse_command(input);

return 0;
}

```

```

adithya@LAPTOP-07A78KDI:~$ wsl
Command 'wsl' not found, but can be installed with:
sudo apt install wsl
adithya@LAPTOP-07A78KDI:~$ cd OS
adithya@LAPTOP-07A78KDI:~/OS$ mkdir Experiment 5
adithya@LAPTOP-07A78KDI:~/OS$ mkdir experiment5
adithya@LAPTOP-07A78KDI:~/OS$ cd expriment5
-bash: cd: expriment5: No such file or directory
adithya@LAPTOP-07A78KDI:~/OS$ nano quotes.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc quotes.c -o quotes
adithya@LAPTOP-07A78KDI:~/OS$ ls
5                Skill6                fifo_client      ls_grep          practical4.c     server
Experiment       Skill7                fifo_client.c    ls_grep.c        practical5       server.c
GIT              Skill8                fifo_server      memory           practical5.c     shell
OSSP             Skill9                fifo_server.c    memory.c         process_demo     shell.c
Skill11          client               file             os-lab           process_demo.c  signal_handler
Skill10          client.c             file.c           output.txt       process_states  signal_handler.c
Skill11          client_to_server     fork             parser           process_states.c source.txt
Skill12          command_history      hello            parser.c         producer_consumer tokenizer
Skill2           command_history.c    hello.c          practical2        producer_consumer.c tokenizer.c
Skill3           dynamic_memory       keyboard_shell   practical2.c     quotes          zombie
Skill4           dynamic_memory.c     keyboard_shell.c practical3.c     quotes.c        zombie.c
Skill5           experiment5          keyboard_shell.c.save practical4       sample.txt

```

```

adithya@LAPTOP-07A78HDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo 'Hello World'

=====
Parsing Result
=====
Normal token: [echo]
Single quoted token: [Hello World]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [Hello World]
adithya@LAPTOP-07A78HDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo '$NAME'

=====
Parsing Result
=====
Normal token: [echo]
Single quoted token: [$NAME]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [$NAME]

```

```

adithya@LAPTOP-07A78HDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo "$NAME"

=====
Parsing Result
=====
Normal token: [echo]
Double quoted token: [Adithya]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [Adithya]
adithya@LAPTOP-07A78HDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo "Hello World"

=====
Parsing Result
=====
Normal token: [echo]
Double quoted token: [Hello World]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [Hello World]

```

```

adithya@LAPTOP-07A78KDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo 'Hello $NAME from $COLLEGE'

=====
Parsing Result
=====
Normal token: [echo]
Single quoted token: [Hello $NAME from $COLLEGE]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [Hello $NAME from $COLLEGE]
adithya@LAPTOP-07A78KDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo "Hello World" 'Good Morning' $NAME

=====
Parsing Result
=====
Normal token: [echo]
Double quoted token: [Hello World]
Single quoted token: [Good Morning]
Normal token: [$NAME]

Total tokens: 4

Parsed tokens:
Token 1 = [echo]
Token 2 = [Hello World]
Token 3 = [Good Morning]
Token 4 = [$NAME]

```

```

adithya@LAPTOP-07A78KDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo 'Special $HOME * ?'

=====
Parsing Result
=====
Normal token: [echo]
Single quoted token: [Special $HOME * ?]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [Special $HOME * ?]
adithya@LAPTOP-07A78KDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

Enter a command:
echo "CSE AI & DS"

=====
Parsing Result
=====
Normal token: [echo]
Double quoted token: [CSE AI & DS]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [CSE AI & DS]
adithya@LAPTOP-07A78KDI:~/OS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = KL University

```

```

=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = ML University

Enter a command:
echo "CSE AI & DS"

=====
Parsing Result
=====
Normal token: [echo]
Double quoted token: [CSE AI & DS]

Total tokens: 2

Parsed tokens:
Token 1 = [echo]
Token 2 = [CSE AI & DS]
adithya@LAPTOP-67A78KD1:~/DS$ ./quotes
=====
QUOTED STRING PARSER
=====

Environment variables:
NAME = Adithya
COURSE = CSE
COLLEGE = ML University

Enter a command:
echo '' ''

=====
Parsing Result
=====
Normal token: [echo]
Single quoted token: []
Double quoted token: []

Total tokens: 3

Parsed tokens:
Token 1 = [echo]
Token 2 = []
Token 3 = []
adithya@LAPTOP-67A78KD1:~/DS$ |

```